

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Constraint-Based Problem Solving (High)

Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze	5
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze	5
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply	5
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze	5
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply	5
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze	5
7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply	5
8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply	5

9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze	5
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze	5

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Constraint-Based Problem Solving Assessment Rubric					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding ; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1) Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.

1. Initial Query Execution Plan

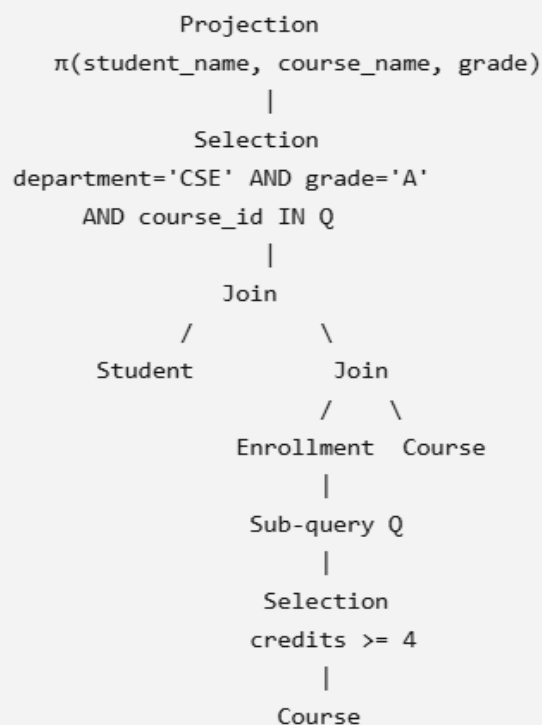
The SQL query can first be represented as a relational algebra expression:

$$\pi_{student_name, course_name, grade} \left(\sigma_{department='CSE' \wedge grade='A' \wedge course_id \in Q} (Student \bowtie Enrollment \bowtie Course) \right)$$

where:

$$Q = \pi_{course_id}(\sigma_{credits \geq 4}(Course))$$

Initial Execution Tree



2. Apply Heuristic Optimization Rules

Heuristic optimization attempts to reduce the amount of data processed at each stage.

Rule 1: Perform selections as early as possible

The original query applies the selection after joining all three tables.

Instead, push the conditions down:

$\sigma_{\text{department}='CSE'}(\text{Student})$

$\sigma_{\text{grade}='A'}(\text{Enrollment})$

$\sigma_{\text{credits} \geq 4}(\text{Course})$

So:

$$S' = \sigma_{\text{department}='CSE'}(\text{Student})$$

$$E' = \sigma_{\text{grade}='A'}(\text{Enrollment})$$

$$C' = \sigma_{\text{credits} \geq 4}(\text{Course})$$

This reduces the number of tuples before joining.

Rule 2: Replace the sub-query with a join

The condition:

```
E.course_id IN (  
  SELECT course_id  
  FROM Course  
  WHERE credits >= 4  
)
```

can be represented using a semijoin or join:

$$E' \bowtie C'$$

or:

$$E' \Join C'$$

Since only courses satisfying credits ≥ 4 are required, we first filter Course.

Rule 3: Push projections down

Only required attributes should be carried through the execution plan.

For Student:

$$S'' = \pi_{\text{student_id}, \text{student_name}}(\sigma_{\text{department}='CSE'}(\text{Student}))$$

For Enrollment:

$$E'' = \pi_{\text{student_id}, \text{course_id}, \text{grade}}(\sigma_{\text{grade}='A'}(\text{Enrollment}))$$

For Course:

$$C'' = \pi_{\text{course_id}, \text{course_name}}(\sigma_{\text{credits} \geq 4}(\text{Course}))$$

This reduces intermediate relation sizes.

Rule 4: Perform the most selective operations first

The optimizer should join the smaller filtered relations first.

Filtered Student

```

|
| student_id
|
JOIN
|

```

Filtered Enrollment

```

|
| course_id
|
JOIN
|

```

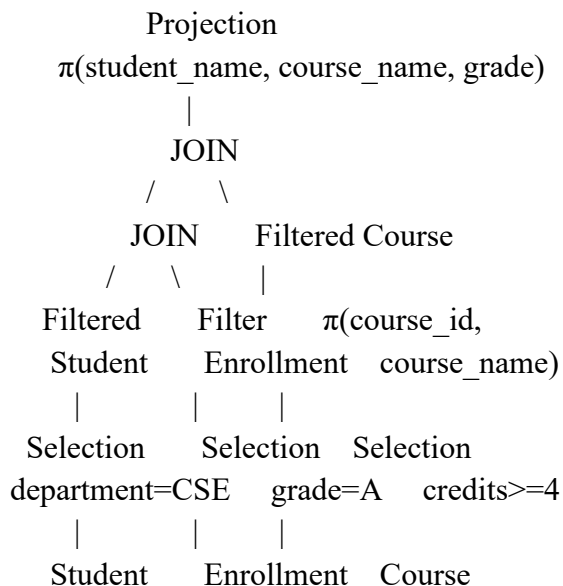
Filtered Course

3. Optimized Query Execution Plan

The optimized relational algebra becomes approximately:

$$\begin{aligned}
 & \pi_{student_name, course_name, grade} \left[\left(\pi_{student_id, student_name} \left(\sigma_{department='CSE'}(Student) \right) \right. \right. \\
 & \quad \bowtie \pi_{student_id, course_id, grade} \left(\sigma_{grade='A'}(Enrollment) \right) \left. \right) \\
 & \quad \bowtie \pi_{course_id, course_name} \left(\sigma_{credits \geq 4}(Course) \right) \left. \right]
 \end{aligned}$$

Optimized Execution Tree



4. Step-by-Step Optimization

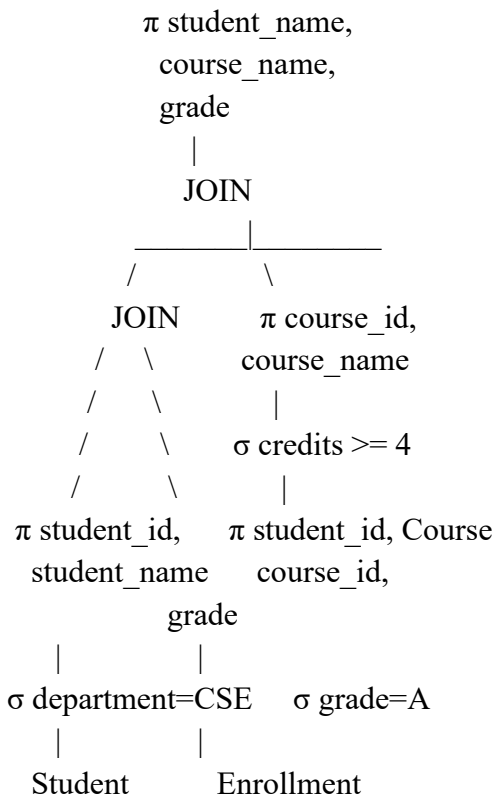
Step	Original Operation	Optimization
1	Join Student, Enrollment, Course	Push selections before joins
2	Filter after joins	Filter Student using department='CSE'
3	Filter Enrollment after join	Filter Enrollment using grade='A'
4	Execute sub-query on Course	Filter Course using credits>=4
5	Carry all attributes	Push projections down
6	Process joins arbitrarily	Join smaller/intermediate relations first

Step Original Operation

Optimization

7 Evaluate IN sub-query separately Transform into semijoin/join

Final Optimized Plan



Main Heuristic Rules Used

1. Perform selections early.
2. Push selections down the query tree.
3. Perform projections early.
4. Push projections down the query tree.
5. Replace nested sub-queries with joins/semijoins where possible.
6. Perform the most selective operations first.
7. Join smaller intermediate relations before larger ones.

2) Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.

1. Calculate Number of Pages

Number of tuples per page:

$$\text{Tuples/page} = \frac{\text{Page size}}{\text{Tuple size}}$$

For Student:

$$\frac{1000}{100} = 10$$

Therefore:

$$B(S) = \frac{10000}{10} = 100 \text{ pages}$$

For Enrollment:

$$\frac{1000}{80} = 12.5$$

Approximately 12 tuples/page:

$$B(E) = \left\lceil \frac{50000}{12} \right\rceil = 4167 \text{ pages}$$

So:

Relation	Tuples	Tuple Size	Pages
Student	10,000	100 B	100
Enrollment	50,000	80 B	4,167

2. Nested-Loop Join

For a block nested-loop join, the smaller relation is used as the outer relation.

Formula:

$$\text{Cost} = B(S) + \left\lceil \frac{B(S)}{M-2} \right\rceil B(E)$$

where:

- $B(S) = 100$
- $B(E) = 4167$
- $M = 101$ buffers

Therefore:

$$\begin{aligned} \text{Cost} &= 100 + \left\lceil \frac{100}{99} \right\rceil (4167) \\ &= 100 + 4167 \\ &= \boxed{4267 \text{ page I/Os}} \end{aligned}$$

3. Sort-Merge Join

For external sorting:

$$SortCost(R) = 2B(R) \left(1 + \left\lceil \log_{M-1} \left(\frac{B(R)}{M} \right) \right\rceil \right)$$

Sort Student

$$B(S) = 100$$

Since 100 pages can almost fit in 101 buffers, the cost is approximately:

$$2(100) = 200$$

Sort Enrollment

$$B(E) = 4167$$

Initial runs:

$$\left\lceil \frac{4167}{101} \right\rceil = 42$$

With 100 pages available for merging, one merge pass is enough.

Therefore:

$$\begin{aligned} Sort(E) &\approx 2(4167) + 2(4167) \\ &= 16668 \end{aligned}$$

Then the final merge/join requires approximately:

$$100 + 4167 = 4267$$

Total:

$$Cost_{SMJ} \approx 200 + 16668 + 4267$$

$$\boxed{21135 \text{ page I/Os}}$$

4. Hash Join

For a standard Grace hash join, assuming enough buffers and a reasonably uniform hash distribution:

$$Cost_{Hash} \approx 3(B(S) + B(E))$$

Therefore:

$$= 3(100 + 4167)$$

$$= 3(4267)$$

$$\boxed{12801 \text{ page I/Os}}$$

5. Compare the Costs

Join Algorithm	Estimated Cost
----------------	----------------

Nested-Loop Join	4,267 I/Os
------------------	------------

Hash Join	12,801 I/Os
-----------	-------------

Sort-Merge Join	21,135 I/Os
-----------------	-------------

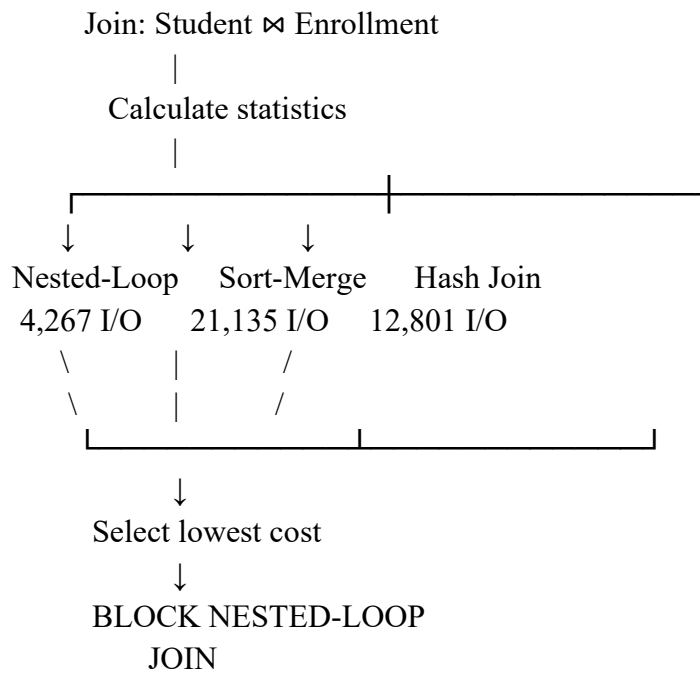
Decision

$$\boxed{\text{Choose Block Nested-Loop Join}}$$

because it has the lowest estimated I/O cost.

6. Cost-Based Optimization Plan

The optimizer would follow these steps:



3) Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with Prepared Statement.

1. Database Setup

Create a database and table in MySQL:

```
CREATE DATABASE college_db;
```

```
USE college_db;
```

```
CREATE TABLE student (  
    student_id INT PRIMARY KEY,  
    student_name VARCHAR(50),  
    department VARCHAR(30),  
    marks INT  
);
```

```
INSERT INTO student VALUES  
(101, 'Arun', 'CSE', 85),  
(102, 'Divya', 'ECE', 90),  
(103, 'Kavin', 'IT', 78);
```

2. Java JDBC Program

Add the MySQL Connector/J JDBC driver to your Java project, then use:

JDBC Connectivity and PreparedStatement Example

```
import java.sql.*;
```

```
public class JDBCExample {
```

```
    // Database details
```

```
    static final String URL = "jdbc:mysql://localhost:3306/college_db";
```

```
    static final String USER = "root";
```

```
    static final String PASSWORD = "";
```

```
    public static void main(String[] args) {
```

```
        // Parameterized SQL query
```

```
        String sql = "SELECT * FROM student WHERE department = ? AND marks >= ?";
```

```
        try {
```

```
            // 1. Establish connection
```

```
            Connection con = DriverManager.getConnection(  
                URL, USER, PASSWORD
```

```
            );
```

```
            System.out.println("Database connected successfully!");
```

```
            // 2. Create PreparedStatement
```

```

PreparedStatement pstmt = con.prepareStatement(sql);

// 3. Set parameter values
pstmt.setString(1, "CSE");
pstmt.setInt(2, 80);

// 4. Execute query
ResultSet rs = pstmt.executeQuery();

// 5. Display results
System.out.println("\nStudents:");

while (rs.next()) {
    System.out.println(
        "ID: " + rs.getInt("student_id") +
        ", Name: " + rs.getString("student_name") +
        ", Department: " + rs.getString("department") +
        ", Marks: " + rs.getInt("marks")
    );
}

// 6. Close resources
rs.close();
pstmt.close();
con.close();

} catch (SQLException e) {
    System.out.println("Database error: " + e.getMessage());
}
}
}

```

3. How PreparedStatement Works

The query contains placeholders:

```
String sql = "SELECT * FROM student WHERE department = ? AND marks >= ?";
```

The ? symbols are replaced using:

```
pstmt.setString(1, "CSE");
```

```
pstmt.setInt(2, 80);
```

So the query effectively searches for:

department = CSE

AND

marks >= 80

Expected output:

Database connected successfully!

Students:

ID: 101, Name: Arun, Department: CSE, Marks: 85

4. Parameterized INSERT Example

PreparedStatement can also be used to insert records:

```
import java.sql.*;
```

```
public class InsertStudent {

    static final String URL = "jdbc:mysql://localhost:3306/college_db";
    static final String USER = "root";
    static final String PASSWORD = "";

    public static void main(String[] args) {

        String sql = "INSERT INTO student " +
            "(student_id, student_name, department, marks) " +
            "VALUES (?, ?, ?, ?)";

        try {
            Connection con = DriverManager.getConnection(
                URL, USER, PASSWORD
            );

            PreparedStatement pstmt = con.prepareStatement(sql);

            pstmt.setInt(1, 104);
            pstmt.setString(2, "Rahul");
            pstmt.setString(3, "CSE");
            pstmt.setInt(4, 88);

            int rows = pstmt.executeUpdate();

            System.out.println(rows + " record inserted successfully.");

            pstmt.close();
            con.close();

        } catch (SQLException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Why use PreparedStatement?

1. Prevents SQL injection by separating SQL code from parameter values.
2. Improves readability of parameterized queries.
3. Allows values to be supplied dynamically.
4. Can improve performance when the same SQL statement is executed multiple times.
5. Provides convenient methods such as `setInt()`, `setString()`, `setDouble()`, etc.

4) Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.

Step 1: Identify conflicting operations

Two operations conflict when:

1. They belong to different transactions.
2. They access the same data item.
3. At least one operation is a write.

Conflicts on A

- T2: R(A) occurs before T1: W(A)
 $T2 \rightarrow T1$
- T1: W(A) occurs before T3: R(A)
 $T1 \rightarrow T3$
- T1: W(A) occurs before T3: W(A)
 $T1 \rightarrow T3$
- T3: W(A) occurs before T2: W(A)
 $T3 \rightarrow T2$

Conflicts on B

- T2: W(B) occurs before T4: R(B)
 $T2 \rightarrow T4$
- T2: W(B) occurs before T4: W(B)
 $T2 \rightarrow T4$
- T4: W(B) occurs before T1: R(B)
 $T4 \rightarrow T1$

Step 2: Construct the precedence graph

The transactions are the vertices:

T1 T2 T3 T4

The edges are:

$T2 \rightarrow T1$

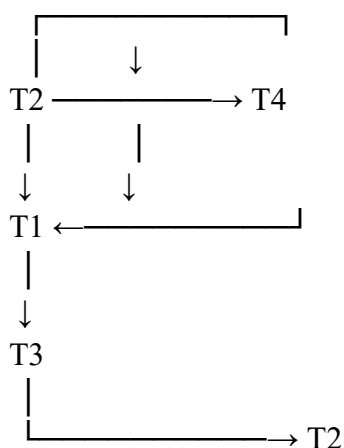
$T1 \rightarrow T3$

$T3 \rightarrow T2$

$T2 \rightarrow T4$

$T4 \rightarrow T1$

Graphically:



Step 3: Check for cycles

We can identify the cycle:

$$T2 \rightarrow T1 \rightarrow T3 \rightarrow T2$$

There is also:

$$T2 \rightarrow T4 \rightarrow T1 \rightarrow T3 \rightarrow T2$$

Therefore, the precedence graph contains a cycle.

Step 4: Conclusion

A schedule is conflict-serializable if and only if its precedence graph is acyclic.

Here:

Cycle exists

5) Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.

Step 1: Apply Basic 2PL

Initially:

A → Unlocked

B → Unlocked

T1

T1 wants to read and write A.

T1: X-lock(A)

T1: Read(A)

T1: Write(A)

T1 now holds:

A → X-lock(T1)

T1 then requests B:

T1: X-lock(B)

If B is currently unlocked, T1 obtains the lock.

A → X-lock(T1)

B → X-lock(T1)

T1 can perform:

Read(B)

Write(B)

After acquiring all required locks, T1 enters its shrinking phase:

T1: Unlock(A)

T1: Unlock(B)

2. Concurrent Execution Causing Deadlock

A deadlock can occur if the transactions acquire resources in opposite order.

Consider this schedule:

Step T1	T2
1 X-lock(A)	
2	X-lock(B)
3 Request X-lock(B) → Wait	
4	Request X-lock(A) → Wait

Now:

T1 holds A

T2 holds B

T1 waits for B

T2 waits for A

Neither transaction can continue.

3. Wait-For Graph

A wait-for graph represents transactions as nodes.

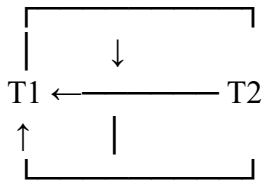
Because T1 is waiting for a lock held by T2:

$T1 \rightarrow T2$

Because T2 is waiting for a lock held by T1:

$$T2 \rightarrow T1$$

Graph:



There is a cycle:

$$T1 \rightarrow T2 \rightarrow T1$$

Therefore, deadlock exists.

4. Deadlock Resolution

One common approach is to abort one transaction.

For example, abort T2:

$T2 \rightarrow \text{ABORT}$

T2 releases its locks:

T2: Unlock(B)

T2: Unlock(A)

Now B becomes available.

T1: X-lock(B)

T1: Read(B)

T1: Write(B)

T1 completes:

T1: COMMIT

T1: Unlock(A)

T1: Unlock(B)

T2 can then be restarted:

T2: X-lock(B)

T2: X-lock(A)

T2: Read(B)

T2: Write(B)

T2: Read(A)

T2: Write(A)

T2: COMMIT

5. Correct 2PL Execution

A better solution is to make both transactions acquire locks in the same order.

For example, always lock A before B.

T1

X-lock(A)

X-lock(B)

Read(A)

Write(A)

Read(B)

Write(B)
 Unlock(A)
 Unlock(B)
 COMMIT
 T2
 X-lock(A) → Wait if T1 holds A

↓
 T1 completes

↓
 X-lock(A)
 X-lock(B)
 Read(B)
 Write(B)
 Read(A)
 Write(A)
 Unlock(A)
 Unlock(B)
 COMMIT

There is no circular wait, so the deadlock is avoided.

6. Important 2PL Rules

Phase	Allowed Operation
-------	-------------------

Growing	Acquire S/X locks
---------	-------------------

Growing	Cannot release locks
---------	----------------------

Shrinking	Release locks
-----------	---------------

Shrinking	Cannot acquire new locks
-----------	--------------------------

Deadlock Detection

Use a wait-for graph:

If graph contains a cycle

↓

Deadlock

↓

Abort/rollback one transaction

↓

Release its locks

↓

Other transaction continues

Final Conclusion

Basic 2PL guarantees conflict serializability by dividing transaction execution into growing and shrinking phases. However, it does not prevent deadlocks. A deadlock occurs when transactions hold locks while waiting for each other's locks, producing a cycle in the wait-for graph. The deadlock can be resolved by aborting/rolling back one transaction or prevented by enforcing a consistent lock-acquisition order.

6) Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.

1. Wait-Die Scheme

Rule

When transaction T_i requests a lock held by T_j :

- If T_i is older than $T_j \rightarrow T_i$ waits
- If T_i is younger than $T_j \rightarrow T_i$ dies (aborts)

In other words:

Older $\rightarrow$ Younger = WAIT

Younger $\rightarrow$ Older = DIE

Step-by-Step

Step 1

T1 obtains lock on A.

$T1 \rightarrow X\text{-lock}(A)$

Step 2

T2 requests A.

T2 is younger than T1:

$$TS(T2) = 10 > TS(T1) = 5$$

Therefore:

$T2 \rightarrow \text{DIE}$

T2 is aborted and its locks, if any, are released.

Step 3

T2 is restarted with its original timestamp.

$T2 \rightarrow \text{Restart}$

It can request its locks again.

Step 4

Suppose T2 obtains B.

$T2 \rightarrow X\text{-lock}(B)$

Step 5

T3 requests B.

T3 is younger than T2:

$$15 > 10$$

Therefore:

$T3 \rightarrow \text{DIE}$

T3 is aborted.

Wait-Die Result

T1 (TS=5) $\rightarrow$ locks A

↑

|

T2 (TS=10) $\rightarrow$ requests A $\rightarrow$ DIE

T2 (TS=10) → locks B

↑
|

T3 (TS=15) → requests B → DIE

No waiting cycle can occur, so deadlock is prevented.

2. Wound-Wait Scheme

Rule

When transaction T_i requests a lock held by T_j :

- If T_i is older than T_j → T_i wounds T_j ; T_j is aborted.
- If T_i is younger than T_j → T_i waits.

In other words:

Older → Younger = WOUND

Younger → Older = WAIT

Step-by-Step

Step 1

T1 obtains A:

T1 → X-lock(A)

Step 2

T2 requests A.

T2 is younger than T1:

$$10 > 5$$

Therefore:

T2 → WAIT

T2 waits for T1 to release A.

Step 3

Suppose T2 obtains B after it becomes available.

T2 → X-lock(B)

Step 4

T3 requests B.

T3 is younger than T2:

$$15 > 10$$

Therefore:

T3 → WAIT

Step 5

Now suppose T1 requests B while T2 holds B.

T1 is older than T2:

$$5 < 10$$

Therefore T1 wounds T2:

T1 → WOUND → T2

T2 is aborted and releases B.

Then T1 obtains B:

$T_1 \rightarrow X\text{-lock}(B)$

3. Comparison

Feature	Wait-Die	Wound-Wait
Type	Non-preemptive	Preemptive
Older requests younger's lock	Wait	Wound/abort younger
Younger requests older's lock	Die/abort	Wait
Transaction rollback	More frequent for younger transactions	Older transactions get priority
Deadlock prevention	Yes	Yes
Waiting	Older transactions may wait	Younger transactions may wait

Easy Way to Remember

Wait-Die:

Older waits, younger dies.

Wound-Wait:

Older wounds, younger waits.

4. Why Deadlock Cannot Occur

Both schemes impose an ordering based on timestamps.

For Wait-Die, a transaction can only wait for an older transaction:

$$T_i \rightarrow T_j \text{ only if } TS(T_i) < TS(T_j)$$

For Wound-Wait, a transaction can only wait for an older transaction:

$$T_i \rightarrow T_j \text{ only if } TS(T_i) > TS(T_j)$$

A circular wait would require timestamps to continuously move in one direction and eventually return to the starting transaction, which is impossible.

7) Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.

Step 1: Identify the Checkpoint

The checkpoint occurs at:

< CHECKPOINT >

After the checkpoint, we examine transactions that were active and transactions that committed.

From the log:

T1 → COMMITTED

T2 → ACTIVE

T3 → COMMITTED

T4 → ACTIVE

Therefore:

Committed transactions

$T1, T3$

Uncommitted transactions

$T2, T4$

Step 2: REDO Committed Transactions

For Immediate Update, changes made by committed transactions must be redone to guarantee that their updates are reflected in the database.

Committed transactions:

T1

T3

REDO T1

Log record:

<T1, A, 100, 150>

Apply:

A = 150

REDO T3

Log record:

<T3, C, 300, 350>

Apply:

C = 350

Step 3: UNDO Uncommitted Transactions

Transactions that were active when the crash occurred must be rolled back.

Uncommitted transactions:

T2

T4

For UNDO, process their log records in reverse order.

UNDO T4

Log:

<T4, D, 400, 450>

Restore the old value:

D = 400

UNDO T2

T2 has:

<T2, B, 250, 275>

<T2, B, 200, 250>

Undo in reverse order.

First:

B = 250

Then:

B = 200

Therefore, final value:

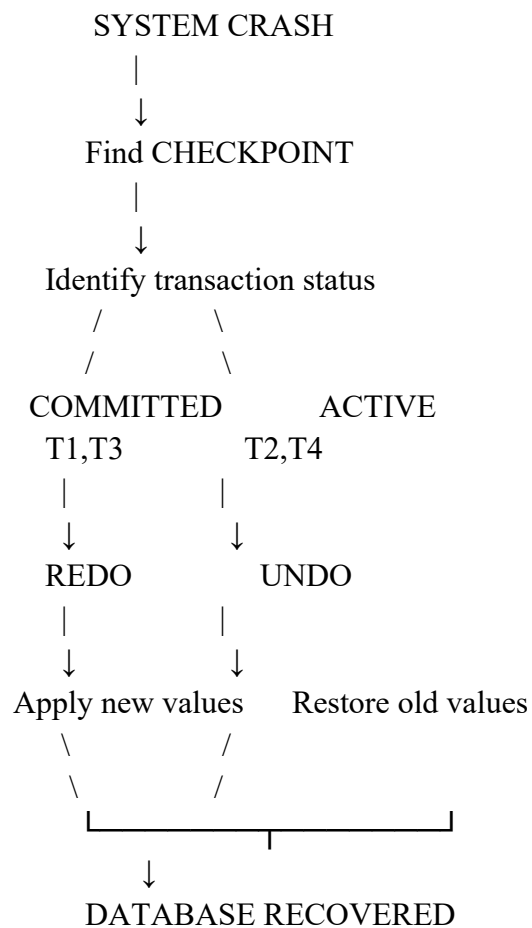
B = 200

Step 4: Recovery Table

Transaction Status at Crash Recovery Action

T1	Committed	REDO
T2	Uncommitted	UNDO
T3	Committed	REDO
T4	Uncommitted	UNDO

Step 5: Recovery Sequence



Final Database State

Assuming the initial values were:

A = 100

B = 200

C = 300

D = 400

After recovery:

Data Item Final Value Reason

A	150	T1 committed → REDO
B	200	T2 uncommitted → UNDO
C	350	T3 committed → REDO
D	400	T4 uncommitted → UNDO

8) Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.

Step 1: Identify Transaction Status

Before the crash:

T1

START T1

UPDATE A

COMMIT T1

Therefore:

$$T1 = \boxed{COMMITTED}$$

T2

START T2

UPDATE B

UPDATE B

No COMMIT record exists.

$$T2 = \boxed{UNCOMMITTED}$$

T3

START T3

UPDATE C

COMMIT T3

Therefore:

$$T3 = \boxed{COMMITTED}$$

T4

START T4

UPDATE D

No COMMIT record exists.

$$T4 = \boxed{UNCOMMITTED}$$

Step 2: Select Transactions for REDO

In Deferred Update recovery:

Only transactions with a COMMIT record are REDOed.

Therefore:

Committed:

T1, T3

Uncommitted:

T2, T4

Recovery actions:

Transaction	Status	Action
T1	Committed	REDO
T2	Uncommitted	Ignore

Transaction	Status	Action
T3	Committed	REDO
T4	Uncommitted	Ignore

Step 3: REDO T1

T1 changed:

<T1, A, 100, 150>

REDO means apply the new value:

$$A = 150$$

Step 4: Ignore T2

T2 did not commit.

Its updates:

<T2, B, 200, 250>

<T2, B, 250, 275>

are not applied to the database.

No UNDO operation is necessary because, under deferred update, these changes were not written to the database before commit.

Therefore:

$$B = 200$$

Step 5: REDO T3

T3 changed:

<T3, C, 300, 350>

REDO:

$$C = 350$$

Step 6: Ignore T4

T4 did not commit.

Its update:

<T4, D, 400, 450>

is ignored.

Therefore:

$$D = 400$$

Step 7: Final Database State

Assume the original database values were:

A = 100

B = 200

C = 300

D = 400

After recovery:

Data Item Final Value Reason

A	150	T1 committed → REDO
B	200	T2 uncommitted → Ignore
C	350	T3 committed → REDO

Data Item Final Value Reason

D 400 T4 uncommitted → Ignore

Thus:

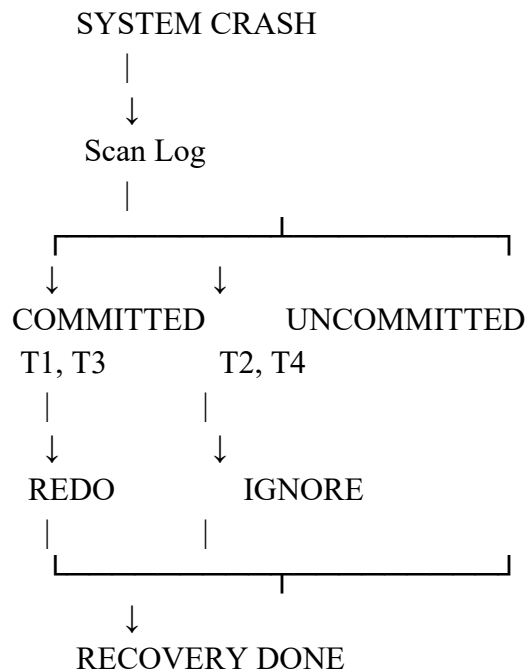
$A = 150, B = 200, C = 350, D = 400$

Recovery Algorithm

DEFERRED_UPDATE_RECOVERY(Log)

1. Scan the log.
2. Identify all transactions having COMMIT records.
3. Mark them as committed.
4. For each committed transaction:
 REDO every update using the new value.
5. For each uncommitted transaction:
 Ignore all its updates.
6. Complete recovery.

Execution



9) Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.

1. Locking Rules

Strict 2PL uses:

- Shared lock (S) → for reading
- Exclusive lock (X) → for writing

Important rule:

All exclusive locks are held until the transaction commits or aborts.

For our banking system:

Operation	Lock
Read balance	S-lock
Update balance	X-lock
Transfer funds	X-lock on both accounts
Commit	Release locks

2. Transaction T1 – Transfer Funds

T1 needs to modify both accounts, so it obtains exclusive locks.

T1:

X-lock(A)
X-lock(B)

Read(A)
Read(B)

$A = A - 1000$
 $B = B + 1000$

Write(A)
Write(B)

COMMIT

Unlock(A)
Unlock(B)

During this entire transaction:

$A \rightarrow X\text{-lock}(T1)$

$B \rightarrow X\text{-lock}(T1)$

No other transaction can read or modify A or B until T1 commits.

3. Transaction T2 – Read Balances

T2 wants to read both balances.

T2:

S-lock(A)

S-lock(B)

Read(A)

Read(B)

COMMIT

Unlock(A)

Unlock(B)

However, if T1 already holds X-locks, T2 cannot acquire the S-locks.

Therefore:

T1: X-lock(A)

T1: X-lock(B)

T2: S-lock(A) → WAIT

T2: S-lock(B) → WAIT

T2 waits until T1 commits.

4. Concurrent Execution

A possible schedule is:

Step T1 – Transfer	T2 – Read
1 X-lock(A)	
2 X-lock(B)	
3 Read(A)	
4 $A = A - ₹1,000$	
5 Write(A)	
6 Read(B)	
7 $B = B + ₹1,000$	
8 Write(B)	
9	S-lock(A) → WAIT
10	S-lock(B) → WAIT
11 COMMIT	
12 Unlock(A), Unlock(B)	
13	S-lock(A)

Step T1 – Transfer	T2 – Read
14	S-lock(B)
15	Read(A), Read(B)
16	COMMIT
17	Unlock(A), Unlock(B)

5. Result

After T1 commits:

Account A = ₹9,000

Account B = ₹6,000

T2 then reads:

A = ₹9,000

B = ₹6,000

T2 never sees:

A = ₹9,000

B = ₹5,000

or

A = ₹10,000

B = ₹6,000

because it cannot read either account while T1's exclusive locks are held.

6. Why Strict 2PL Is Suitable

Without proper locking, T2 could read the accounts while T1 is halfway through the transfer:

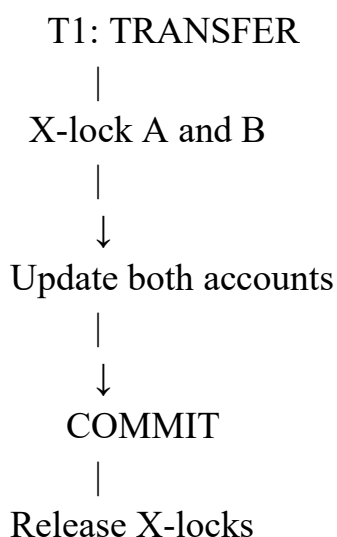
A = ₹9,000

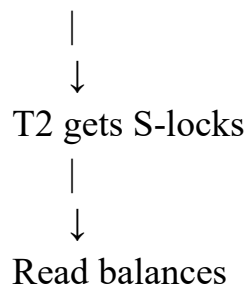
B = ₹5,000

This would make it appear that ₹1,000 has disappeared.

Strict 2PL prevents this dirty/inconsistent read.

Mechanism





10) Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.

Pseudocode: Heuristic Query Optimization

The following pseudocode applies three major heuristic optimization rules:

1. Push selections down as close to base tables as possible.
2. Perform projections early to reduce the number of attributes.
3. Order joins so that smaller intermediate results are generated first.

Pseudocode

ALGORITHM HeuristicQueryOptimization(Query)

INPUT:

Query containing SELECT, FROM, WHERE, JOIN, and PROJECT operations

OUTPUT:

Optimized query execution plan

BEGIN

// Step 1: Convert SQL query into a relational algebra tree

QueryTree $\leftarrow$ BuildRelationalAlgebraTree(Query)

// Step 2: Push Selection Operations Down

FOR each Selection $\sigma(\text{condition})$ in QueryTree DO

IF condition refers only to one base relation THEN

Move $\sigma(\text{condition})$ directly above that relation

ELSE IF condition contains multiple conditions THEN

Split condition into individual conditions

FOR each individual condition DO

Move condition as close as possible to
the corresponding base relation

END FOR

END IF

END FOR

// Step 3: Perform Projection Early

FOR each relation R in QueryTree DO

RequiredAttributes $\leftarrow$ FindAttributesRequiredBy:

- SELECT clause
- JOIN conditions
- WHERE conditions
- GROUP BY / ORDER BY if present

Add Projection operation:

$\pi(\text{RequiredAttributes})(R)$

Remove unnecessary attributes as early as possible

END FOR

// Step 4: Estimate Intermediate Result Sizes

FOR each relation or sub-tree R in QueryTree DO

 EstimateSize(R)

END FOR

// Step 5: Order Joins

JoinList $\leftarrow$ ExtractAllJoins(QueryTree)

WHILE JoinList is not empty DO

 Select the join that produces
 the smallest estimated intermediate result

 Execute/Place selected join in the plan

 Update the estimated size of the
 resulting intermediate relation

 Remove selected join from JoinList

END WHILE

// Step 6: Construct the optimized execution tree

OptimizedTree $\leftarrow$ BuildOptimizedTree(
 SelectionPushDown,
 EarlyProjection,
 JoinOrdering
)

RETURN OptimizedTree

END

Example

Suppose the query is:

```
SELECT S.name, C.course_name
FROM Student S
JOIN Enrollment E ON S.id = E.student_id
JOIN Course C ON E.course_id = C.id
WHERE S.department = 'CSE'
      AND C.credits >= 4;
```

Original Plan

```
      PROJECT
      |
      SELECT
      |
      JOIN
    /  \
  JOIN  Course
 /  \
```

Student Enrollment

This plan may process many unnecessary tuples and attributes.

Step 1: Push Selection Down

The conditions are moved toward their respective tables:

```
σ department='CSE'    σ credits>=4
    |                  |
  Student            Course
```

Instead of filtering after joining, the DBMS filters the tables first.

Step 2: Project Early

Keep only required attributes:

Student:

$\pi(\text{id}, \text{name})$

Enrollment:

$\pi(\text{student_id}, \text{course_id})$

Course:

$\pi(\text{id}, \text{course_name})$

Unnecessary columns are removed before joins.

Step 3: Order Joins

Estimate the sizes:

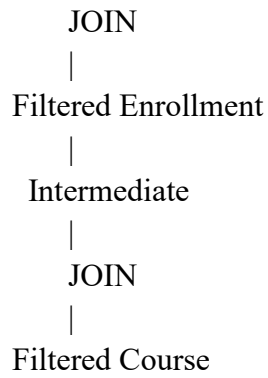
Filtered Student → 500 tuples

Filtered Enrollment → 2,000 tuples

Filtered Course → 100 tuples

The optimizer chooses joins that produce smaller intermediate results.

```
      Filtered Student
      |
```



Optimized Execution Plan

